

RELATÓRIO DE BANCO DE DADOS

Sistema de Controle de Vendas

Relatório sobre tabelas, relacionamentos, views, procedures e triggers

Relatório de Banco de Dados – Sistema de Controle de Vendas
Marcelo Rogério Donadel

08/10/2026

1. Introdução

Este relatório apresenta o banco de dados desenvolvido para uma empresa que precisava organizar melhor suas informações de vendas. Antes da criação do banco, os dados estavam espalhados em planilhas, o que podia dificultar consultas, aumentar a chance de erros e deixar a busca por informações mais demorada. A ideia do projeto foi criar uma estrutura usando SQL.

O banco foi pensado para guardar informações sobre clientes, produtos e pedidos. Com essas informações organizadas, a empresa consegue consultar quem mais compra, qual produto teve mais vendas, quanto cada cliente gastou e quais pedidos passaram de um determinado valor. Além disso, o banco pode servir como base para um sistema de vendas usado por funcionários da empresa.

O sistema também foi montado levando em consideração a organização dos dados. Cada tipo de informação fica em sua própria tabela e os relacionamentos são feitos por meio de chaves. Dessa forma, não é necessário repetir os mesmos dados várias vezes.

Durante o projeto foram utilizados recursos do SQL como tabelas, chaves primárias, chaves estrangeiras, consultas com JOIN, views, procedures e triggers. Cada recurso possui uma função diferente, mas todos ajudam a deixar o banco mais organizado e mais útil para uma aplicação real.

2. Tipo de banco de dados utilizado

O banco utilizado é do tipo relacional. Um banco de dados relacional organiza as informações em tabelas que possuem linhas e colunas. As tabelas podem se relacionar umas com as outras por meio de chaves. Neste projeto, por exemplo, a tabela de pedidos possui uma chave estrangeira que aponta para o cliente que fez aquele pedido.

Esse modelo foi escolhido porque combina bem com o problema apresentado. A empresa possui informações que se relacionam naturalmente: um cliente pode fazer vários pedidos e um produto pode aparecer em vários pedidos. Com um banco relacional, essas relações podem ser representadas de uma maneira simples e organizada.

Outra vantagem é que o SQL permite fazer consultas usando filtros, agrupamentos e relacionamentos entre tabelas. Isso facilita a criação de relatórios e análises de vendas.

3. Por que usar um banco relacional?

- Organiza os dados em estruturas separadas e fáceis de entender.
- Permite relacionar clientes, produtos e pedidos.
- Evita a repetição desnecessária de informações.
- Facilita consultas e relatórios.
- Possui mecanismos de segurança e integridade, como chaves primárias e estrangeiras.
- Pode ser utilizado como base para aplicações reais.

4. Estrutura das tabelas

O banco foi dividido em três tabelas principais: clientes, produtos e pedidos. Essa divisão ajuda a manter cada informação no lugar certo e também segue uma ideia de normalização. Em vez de colocar nome do cliente, telefone, produto e preço dentro de cada pedido, esses dados ficam armazenados nas tabelas correspondentes.

Tabela clientes

A tabela clientes armazena os dados básicos das pessoas que realizam compras. Ela possui o campo id_cliente como chave primária, que identifica cada cliente de forma única.

- id_cliente: identifica o cliente.
- nome: armazena o nome do cliente.
- email: armazena o endereço de e-mail.
- telefone: armazena o número de telefone.

O campo de e-mail foi definido como UNIQUE, para evitar que dois clientes sejam cadastrados com o mesmo endereço.

Tabela produtos

A tabela produtos guarda as informações dos produtos disponíveis para venda. A chave primária é id_produto.

- id_produto: identifica o produto.
- nome: nome do produto.
- categoria: categoria à qual o produto pertence.
- preco: preço atual do produto.
- estoque: quantidade disponível em estoque.

Separar os produtos em uma tabela própria facilita alterações. Por exemplo, se o preço de um mouse mudar, é possível alterar somente o registro do produto.

Tabela pedidos

A tabela pedidos registra as compras realizadas. Ela possui id_pedido como chave primária e também possui as chaves estrangeiras id_cliente e id_produto.

- id_pedido: identifica o pedido.
- id_cliente: indica qual cliente realizou a compra.
- id_produto: indica qual produto foi comprado.
- quantidade: quantidade de unidades compradas.
- data_pedido: data em que a compra foi realizada.

As chaves estrangeiras fazem a ligação entre as tabelas. Assim, o banco consegue saber quem comprou e qual produto foi comprado. Essa ligação também ajuda a impedir referências para clientes ou produtos que não existem.

5. Views utilizadas no banco

As views são consultas SQL que ficam salvas no banco e podem ser acessadas como se fossem tabelas. Elas são úteis quando uma consulta é usada várias vezes. No projeto foram criadas três views para responder às principais perguntas sobre as vendas.

View de clientes que realizaram mais compras

A view vw_clientes_mais_compras reúne o nome dos clientes e a quantidade de pedidos feitos por cada um. Ela utiliza a função COUNT para contar os pedidos e faz um JOIN entre clientes e pedidos.

```
SELECT * FROM vw_clientes_mais_compras
ORDER BY quantidade_compras DESC;
```

Com essa view, a empresa consegue visualizar rapidamente quais clientes fizeram mais compras. Isso pode ser útil para analisar o comportamento dos consumidores e identificar clientes que compram com mais frequência.

View do produto mais vendido

A view vw_produto_mais_vendido mostra a quantidade total de unidades vendidas de cada produto durante o período definido no projeto. A função SUM é usada para somar as quantidades dos pedidos.

```
SELECT * FROM vw_produto_mais_vendido
ORDER BY quantidade_vendida DESC
LIMIT 1;
```

A consulta acima ordena os produtos do maior para o menor número de unidades vendidas e usa LIMIT 1 para mostrar somente o primeiro colocado.

View do total de vendas por cliente

A view vw_total_vendas_cliente calcula quanto cada cliente gastou. Para isso, é feita uma conta entre a quantidade comprada e o preço do produto. Depois, os valores são somados para cada cliente.

```
SELECT * FROM vw_total_vendas_cliente
ORDER BY total_vendas DESC;
```

Essa informação pode ajudar a empresa a entender quais clientes representam maior valor em vendas.

6. Procedures

Stored procedure, ou simplesmente procedure, é um conjunto de comandos SQL que fica armazenado no banco de dados e pode ser executado quando necessário. Em vez de escrever os mesmos comandos várias vezes, a aplicação pode chamar a procedure e passar os parâmetros necessários.

No banco desenvolvido, as procedures podem ser usadas para facilitar tarefas comuns, como cadastrar um pedido, consultar as vendas de um cliente ou atualizar o estoque. Elas também ajudam a centralizar regras do sistema.

Um exemplo de procedure que poderia ser utilizada neste projeto é uma procedure para consultar as vendas de um cliente:

```
DELIMITER //

CREATE PROCEDURE sp_vendas_cliente(IN cliente_id INT)
BEGIN
    SELECT
        c.nome AS cliente,
        SUM(p.quantidade * pr.preco) AS total_vendas
    FROM clientes c
    INNER JOIN pedidos p
        ON c.id_cliente = p.id_cliente
    INNER JOIN produtos pr
        ON p.id_produto = pr.id_produto
    WHERE c.id_cliente = cliente_id
    GROUP BY c.id_cliente, c.nome;
END //

DELIMITER ;
```

Depois de criada, ela poderia ser chamada informando o código do cliente. A principal vantagem é que a consulta fica salva no banco e pode ser reutilizada sem precisar escrever todo o SELECT novamente.

7. Triggers

Trigger é um mecanismo do banco de dados que executa automaticamente uma ação quando acontece determinado evento em uma tabela. Os eventos mais comuns são INSERT, UPDATE e DELETE.

No sistema de vendas, uma trigger pode ser útil principalmente para controlar o estoque. Quando um pedido é inserido, por exemplo, o banco pode diminuir automaticamente a quantidade do produto no estoque. Isso evita que o funcionário tenha que fazer essa alteração manualmente.

Um exemplo de trigger para diminuir o estoque depois de inserir um pedido seria:

```
DELIMITER //

CREATE TRIGGER trg_atualiza_estoque
AFTER INSERT ON pedidos
FOR EACH ROW
```

```
BEGIN
    UPDATE produtos
    SET estoque = estoque - NEW.quantidade
    WHERE id_produto = NEW.id_produto;
END //

DELIMITER ;
```

Nesse exemplo, NEW representa os dados que acabaram de ser inseridos no pedido. A trigger pega a quantidade comprada e diminui essa quantidade do estoque do produto correspondente.

Triggers são úteis porque ajudam a manter os dados atualizados automaticamente. Porém, devem ser usadas com cuidado, já que uma ação simples pode causar outras alterações no banco sem que o usuário execute essas alterações diretamente.

8. Normalização e integridade dos dados

A normalização é uma técnica usada para organizar um banco de dados e diminuir a repetição de informações. Neste projeto, cada tabela possui uma função específica. Os dados dos clientes ficam em clientes, os dados dos produtos ficam em produtos e as informações das compras ficam em pedidos.

Essa organização evita problemas que poderiam acontecer se tudo estivesse em uma única tabela. Por exemplo, se o telefone de um cliente estivesse repetido em vários pedidos, seria necessário alterar vários registros quando o telefone mudasse. Com a estrutura atual, o telefone fica cadastrado uma vez na tabela de clientes.

As chaves primárias também são importantes para a integridade. Cada registro possui um identificador único. Já as chaves estrangeiras garantem o relacionamento entre as tabelas.

9. Consultas e resultados esperados

Com a estrutura criada, o banco consegue responder às perguntas propostas no projeto. A primeira consulta mostra quais clientes fizeram mais compras. A segunda identifica o produto com maior quantidade vendida no período. A terceira apresenta o valor total gasto por cada cliente. Também é possível consultar pedidos acima de um valor definido.

Essas consultas mostram uma das principais vantagens de utilizar um banco de dados centralizado: as informações podem ser analisadas rapidamente sem precisar procurar manualmente em várias planilhas.

10. Conclusão

O desenvolvimento deste banco de dados mostrou como o SQL pode ser usado para organizar informações de uma empresa. A divisão em tabelas tornou os dados mais fáceis de entender e os relacionamentos permitiram conectar clientes, produtos e pedidos.

As views facilitaram a criação de consultas que podem ser usadas várias vezes. As procedures podem guardar operações importantes e as triggers podem automatizar ações, como a atualização do estoque. Juntos, esses recursos deixam o banco mais preparado para ser utilizado por uma aplicação.

Na minha visão, a principal vantagem desse projeto é que ele transforma informações que poderiam ficar desorganizadas em uma estrutura que pode ser consultada e atualizada de forma mais segura. Mesmo sendo um projeto simples, a estrutura criada já apresenta conceitos usados em sistemas reais de vendas.

Assim, o banco atende ao objetivo inicial de centralizar os dados e permitir análises mais rápidas. Também serve como uma base para futuras melhorias, como cadastro de funcionários, formas de pagamento, histórico de estoque e relatórios mais detalhados.